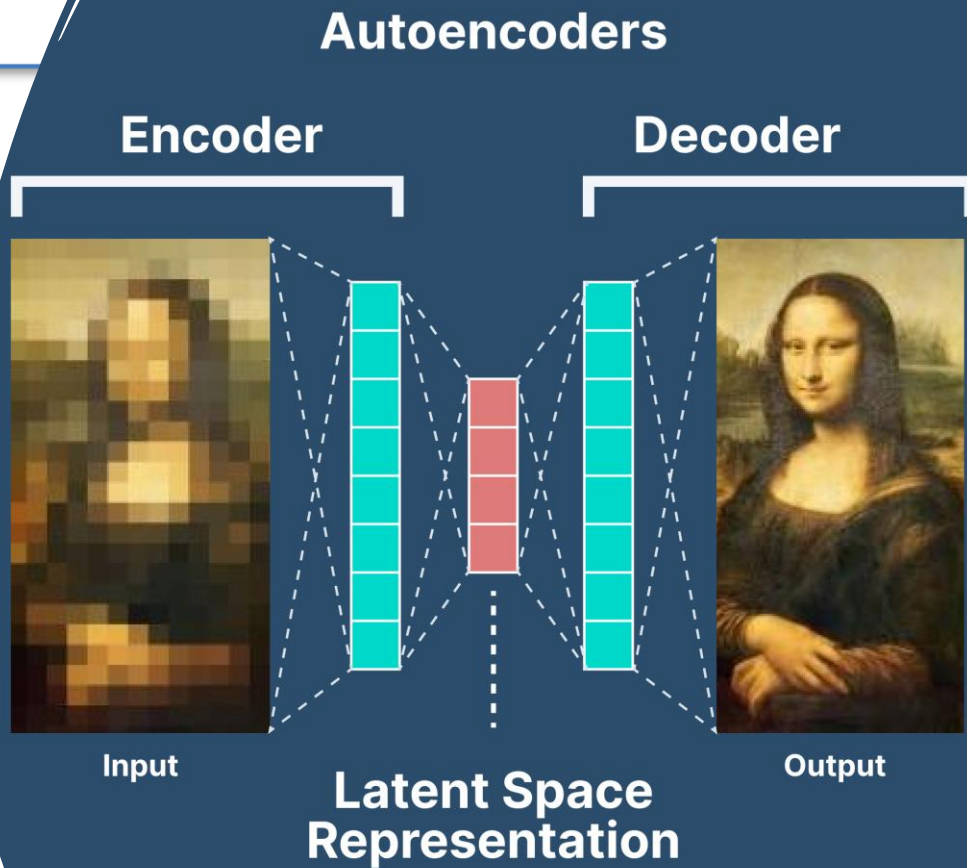


Autoencoders

- Unsupervised learning
- Representation learning
 - **Uncover hidden features**
 - **Enhance interpretability**
 - **Aid in transfer learning**(building models that learn from new classes.)



What is an Autoencoder

Create an architecture with a bottleneck,
which ensures a lower-dimensional
representation of the original data.

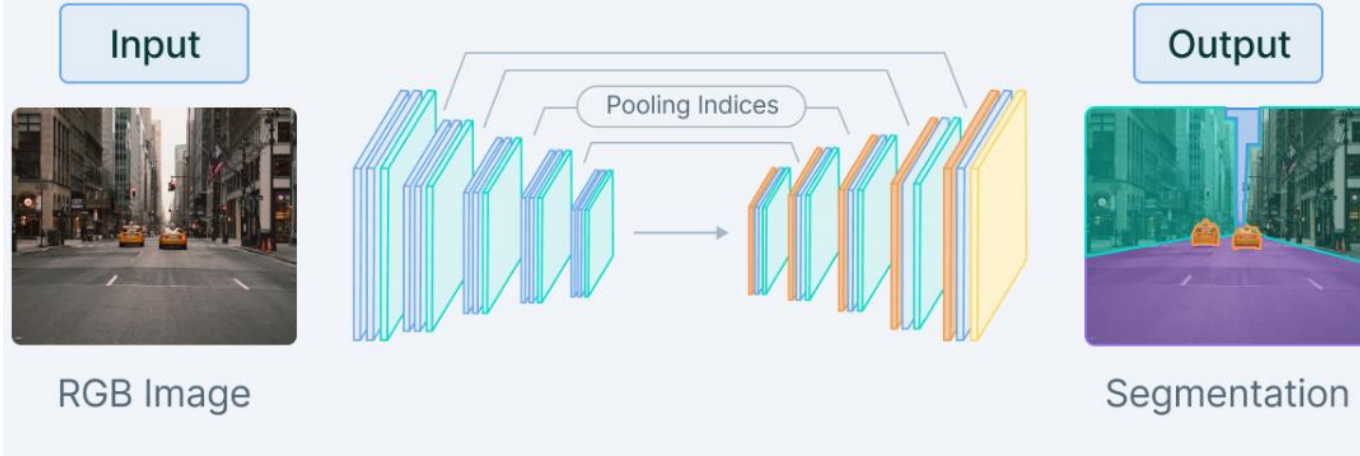
It is one of the technologies and techniques
that enable computer systems to solve **data
compression** problems more efficiently.

➔ solution for reducing noisy data.

Autoencoders: The idea

Create an architecture with a bottleneck,
which ensures a lower-dimensional
representation (encoding) of the original
data.

Convolutional encoder-decoder



Reduce dimensionality → Training the network to find patterns

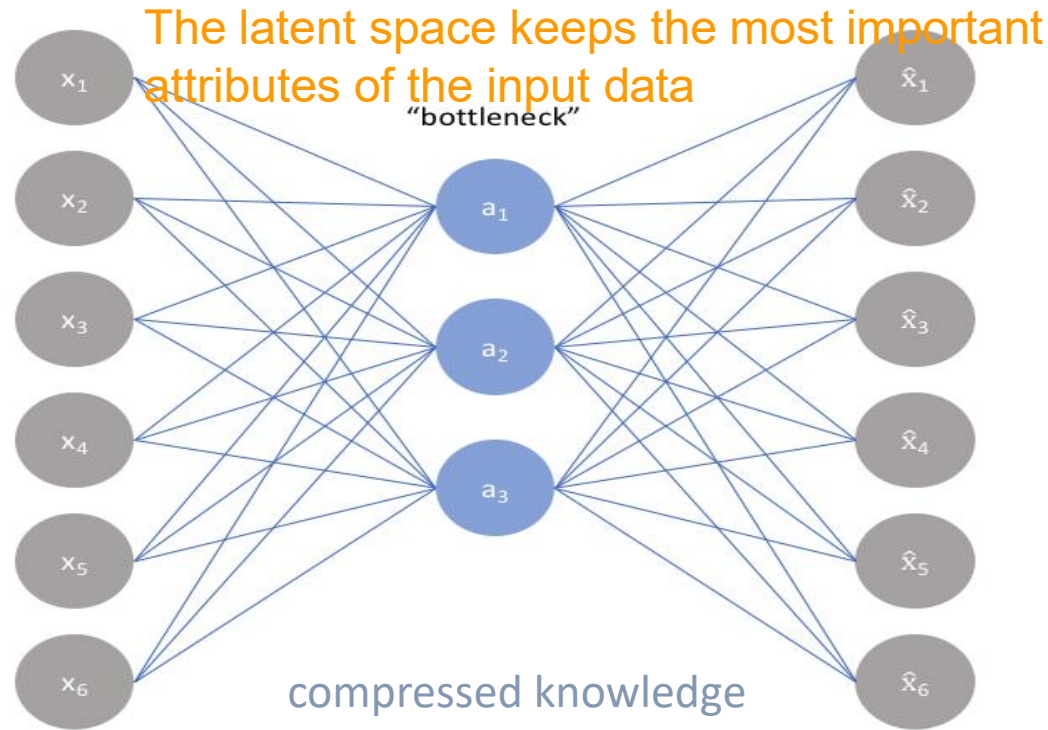
Applications →

- Generation
- Denoising
- Anomaly detection

Input layer

Hidden layer

Output layer

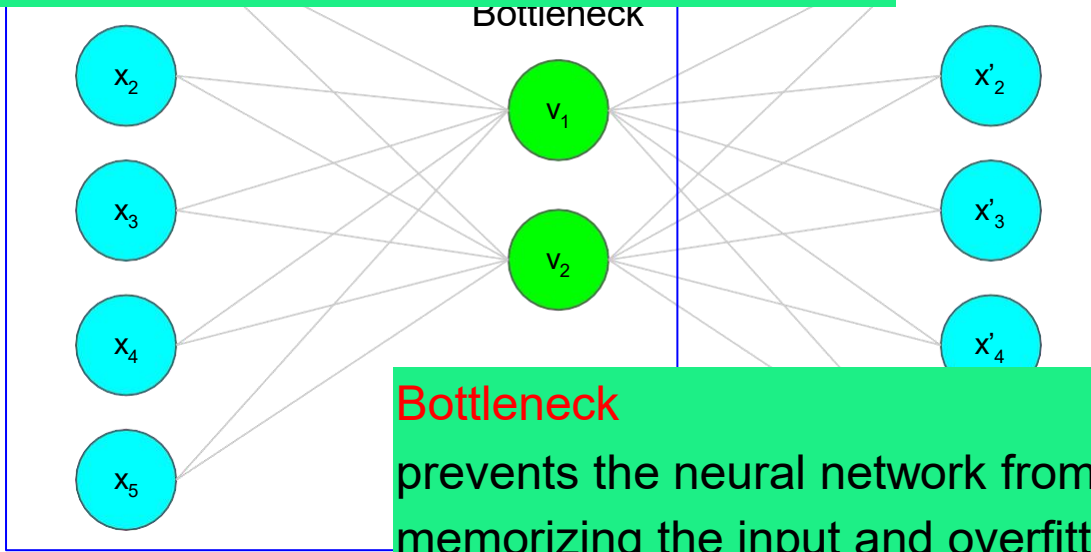


Compresses the train-validate-test set input data

Reconstructs the data back from its encoded form

CONDITION:

Data should have dependencies across dimensions



Output is expected to be the **same** as the input data with reduced noise.

Bottleneck

prevents the neural network from memorizing the input and overfitting on the data → the smaller bottleneck the less overfitting

Encoder = compress data in

512 x 512

256 x 256

128 x 128

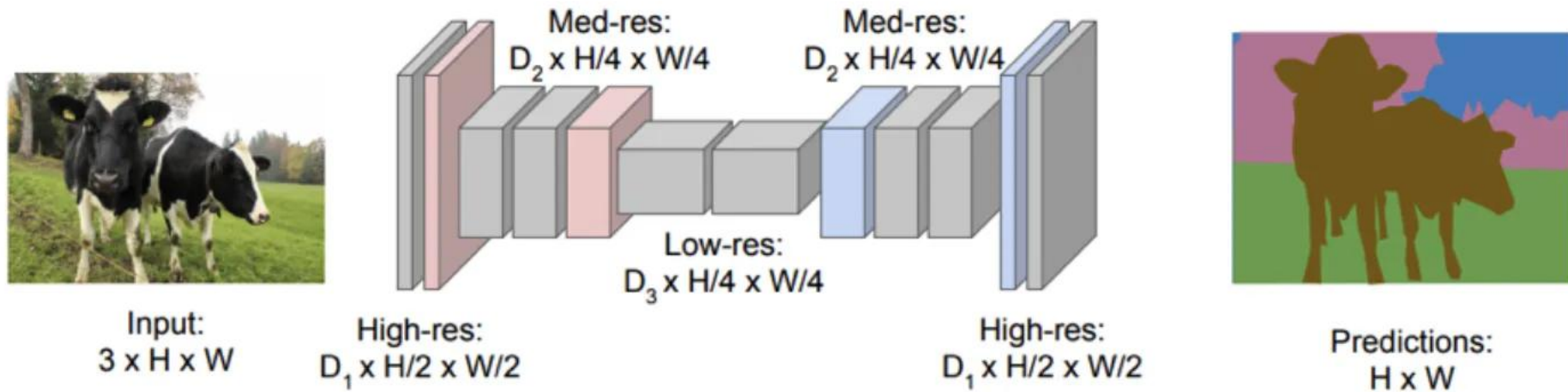
64 x 64

32 x 32

16 x 16

Downsampling

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



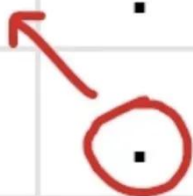
Solution: Make network deep and *work at a lower spatial resolution* for many of the layers.

Up sampling by nearest neighbor interpolation



5	9		
16	25		
5	.	.	9
.	.	.	.
.	.	.	.
16	.	.	25

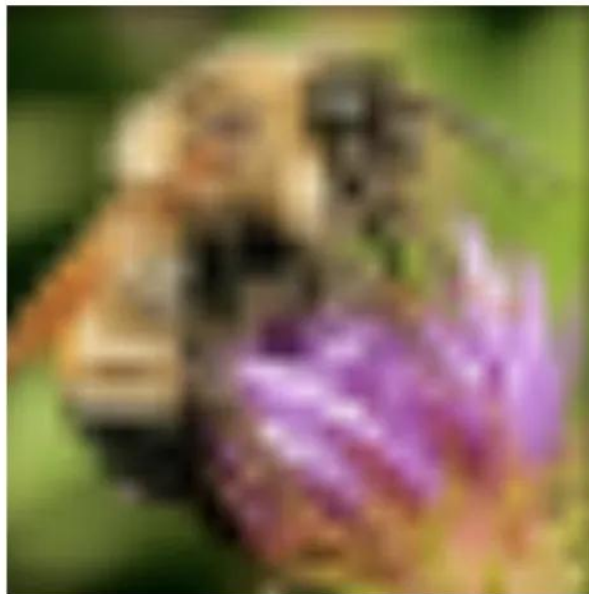
5	5	9	9
5	5	9	9
16	16	25	25
16	16	25	25



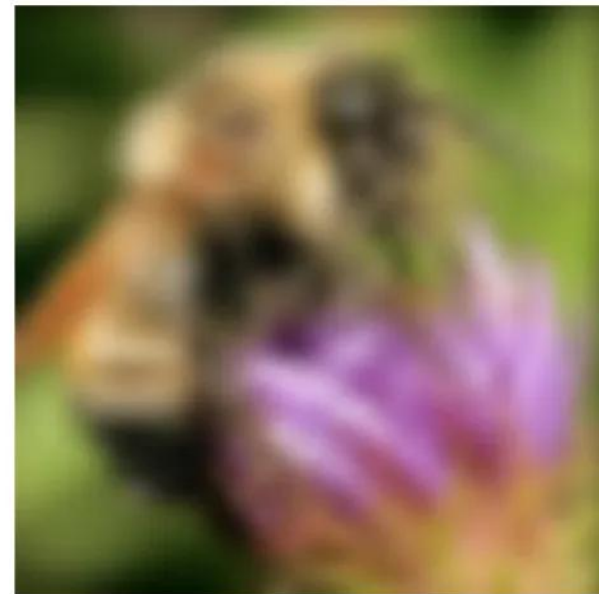
An image multiplied by a factor of 10 using different techniques (ordered by growing complexity).



Nearest-neighbor interpolation



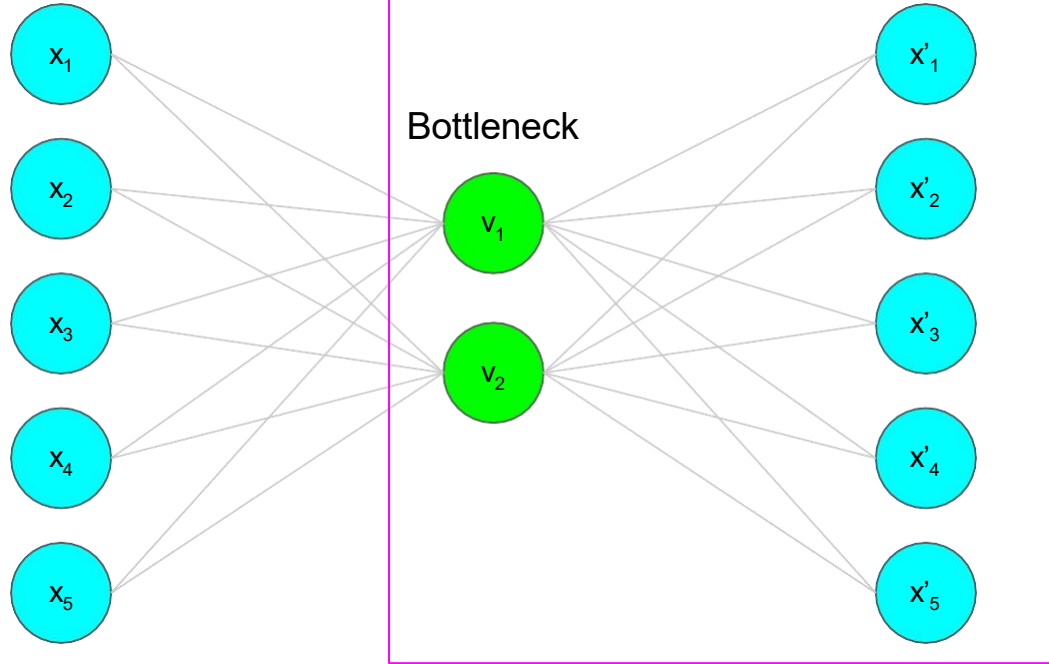
Bilinear interpolation



Bicubic interpolation

Comparison of the upsampled images' resolution

Input layer



Output layer

Bottleneck

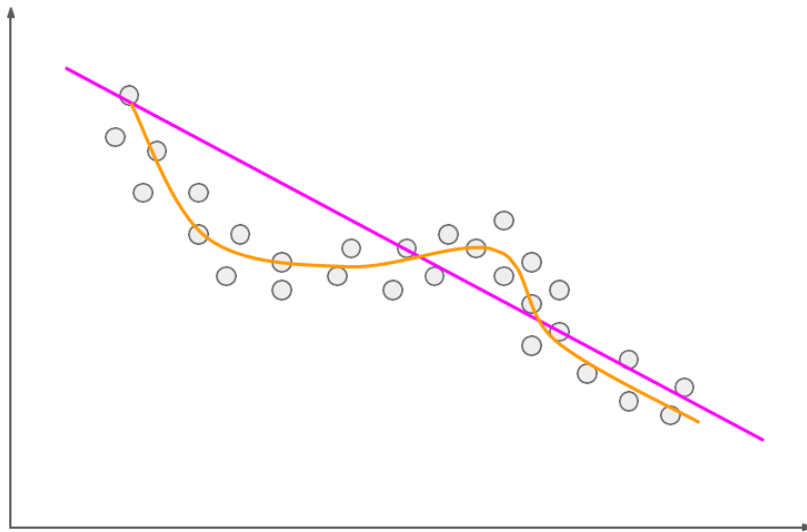
Decoder = Decompress representation back to original domain

6 types of autoencoders

1. Undercomplete (unsupervised/simple) autoencoders
2. Sparse autoencoders
3. Contractive autoencoders
4. Denoising autoencoders
5. Variational Autoencoders (for generative modelling)
6. Deep Autoencoder

PCA vs Encoders

- Both perform dimensionality reduction
- PCA learns linear relationships
- Encoders can learn non-linear relationships
- Encoder = PCA, if it uses linear activation functions



How can we train an autoencoder?

- Backpropagation
- Minimise reconstruction error

$$E\left(\boxed{x}, \boxed{\hat{x}}\right)$$

Original data Reconstructed data

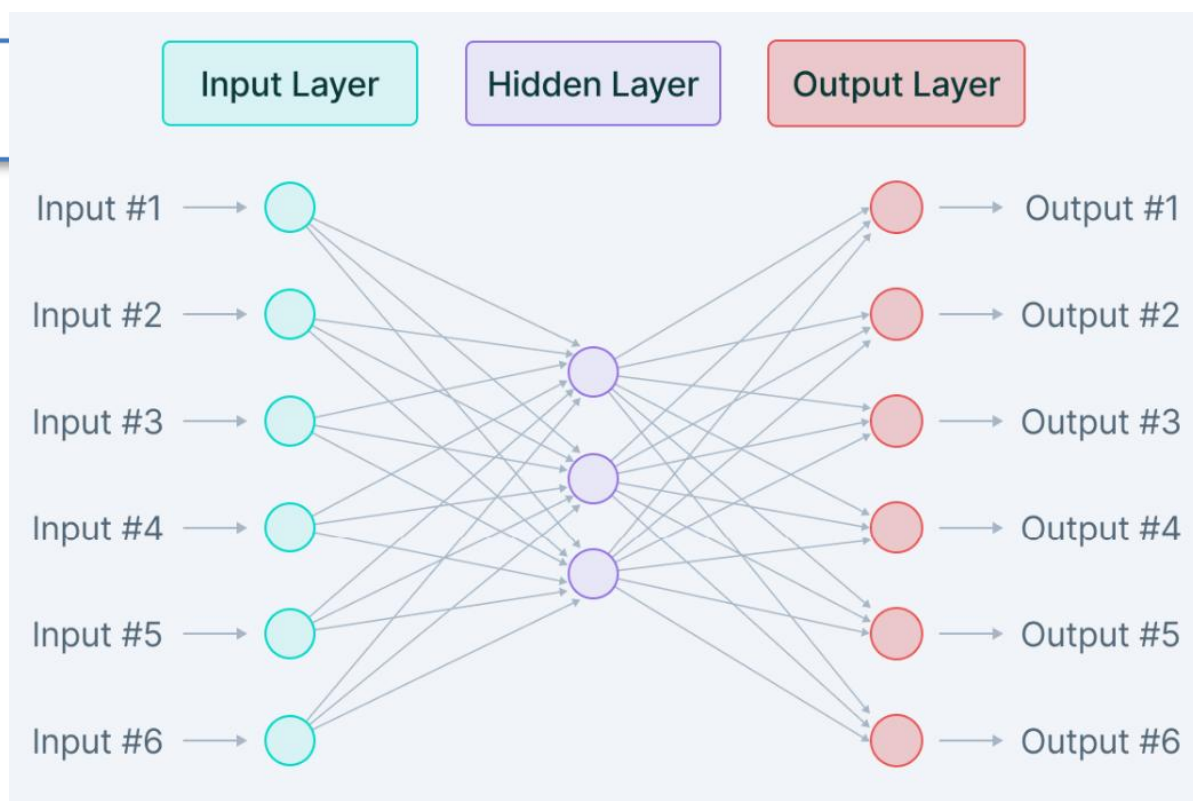
2. Sparse autoencoders

- Sensitive enough to input data to reconstruct it
- Insensitive enough to input data **not** to overfit it

The loss function

$$E(x, \hat{x}) + regularization$$

Sparse autoencoders work by penalizing the activation of some neurons in hidden layers (**not on** the size of the weights at the nodes)



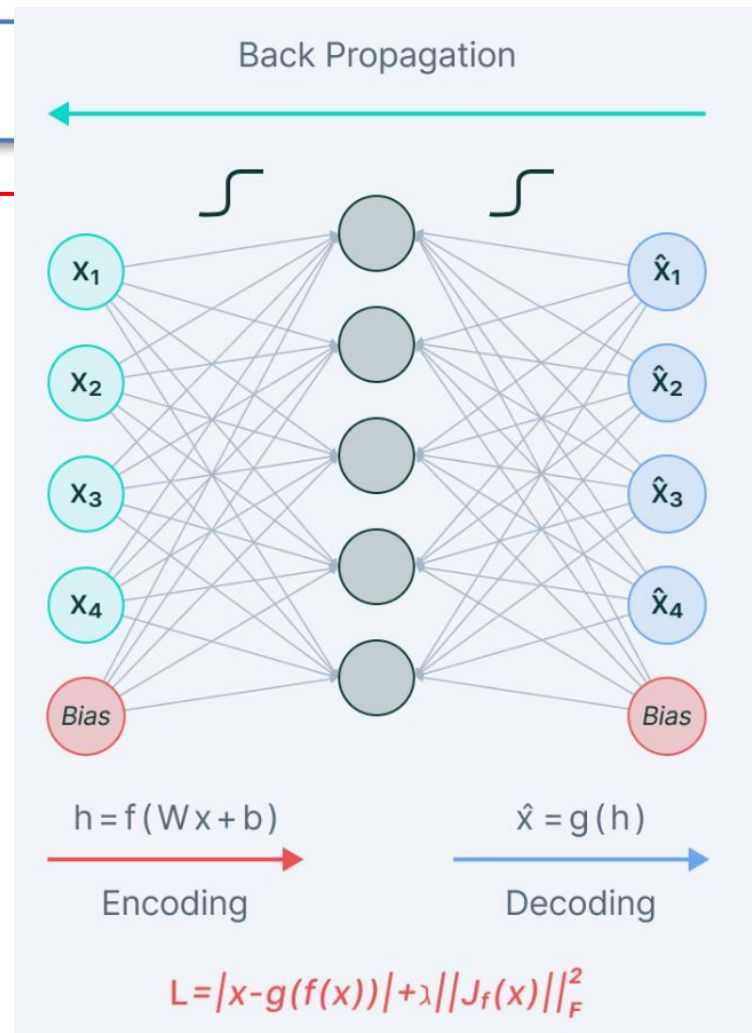
$$L = x - \hat{x} + \lambda \sum_i a_i^{(h)}$$

Where h represents the hidden layer, i represents the image in the minibatch, and a represents the activation

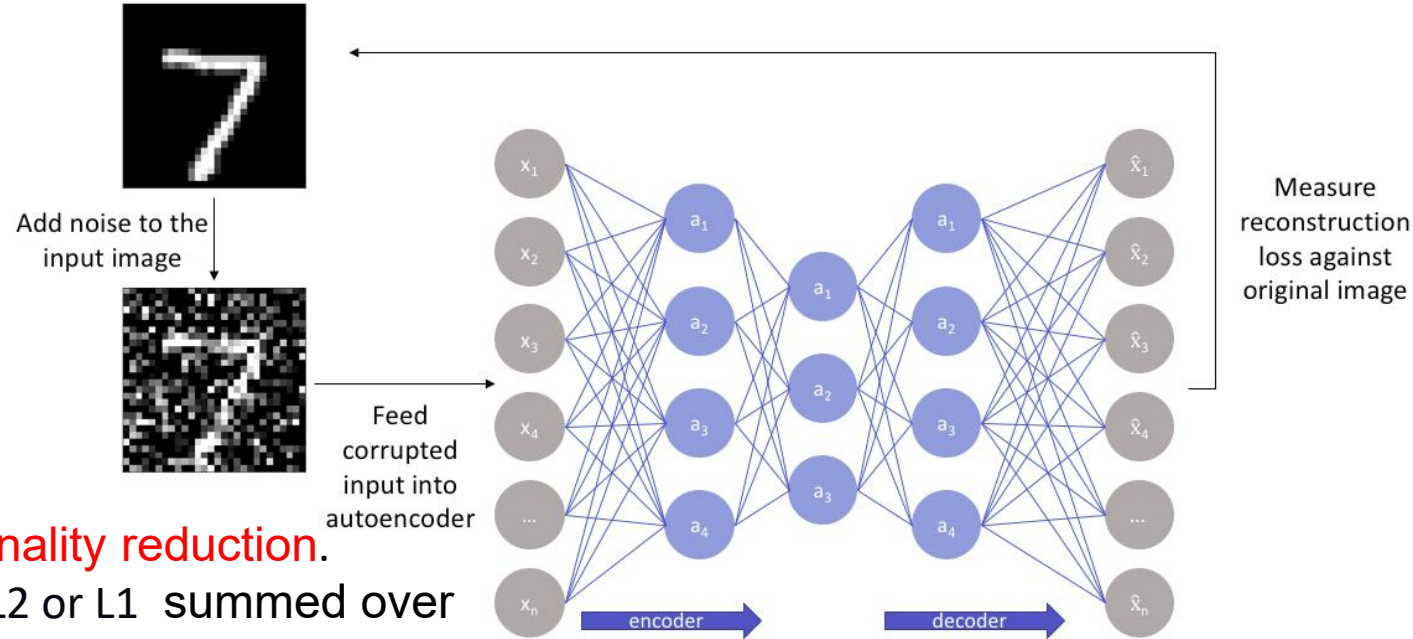
3. Contractive autoencoders

Latent space does not vary by a huge amount for minor variations in the input

Gradient is summed over all training samples



4. Denoising autoencoders

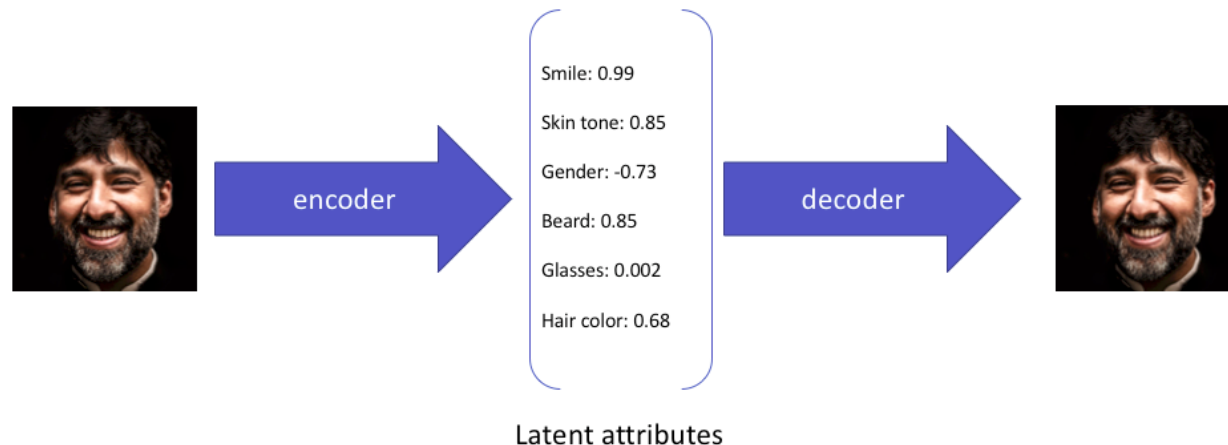


Non-linear dimensionality reduction.

The loss function is L2 or L1 summed over all training samples

Does not use the input as a ground truth

5. Variational autoencoders



Identify the characteristics of the latent vector z that reconstructs the output given a particular input → estimate parametrized distribution → minimize the KL divergence between the original distribution and our parameterized one.

bypass the sampling process from the neural network

Loss function: Variational Autoencoder



Reconstruction error

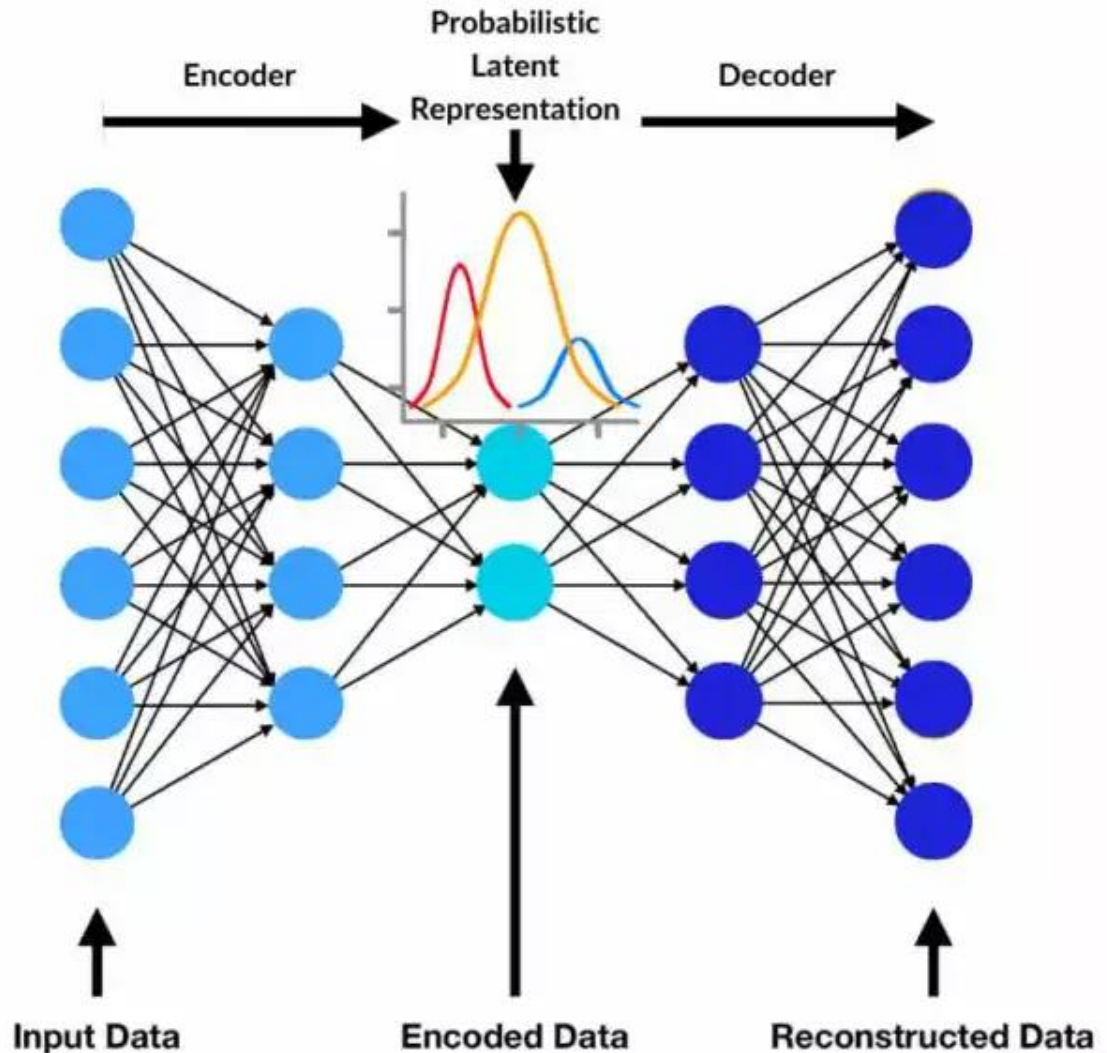
Kullback–Leibler Divergence

$$LOSS = RMSE + KL$$

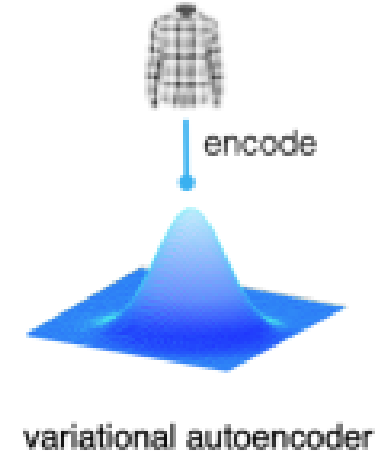
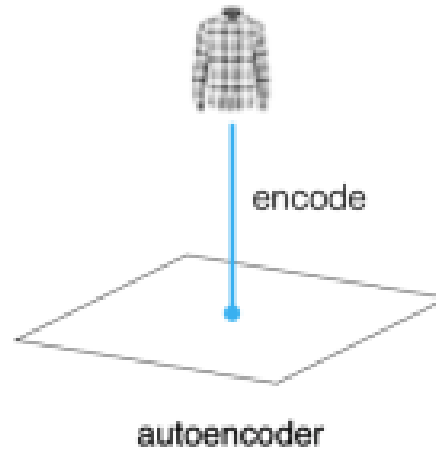
Penalize observations where mean and log variance vectors differ significantly from the parameters of a standard normal distribution (mean vector = 0 and log variance = 0)

Difference between normal distribution (mean vector, log variance vector) from standard normal distribution

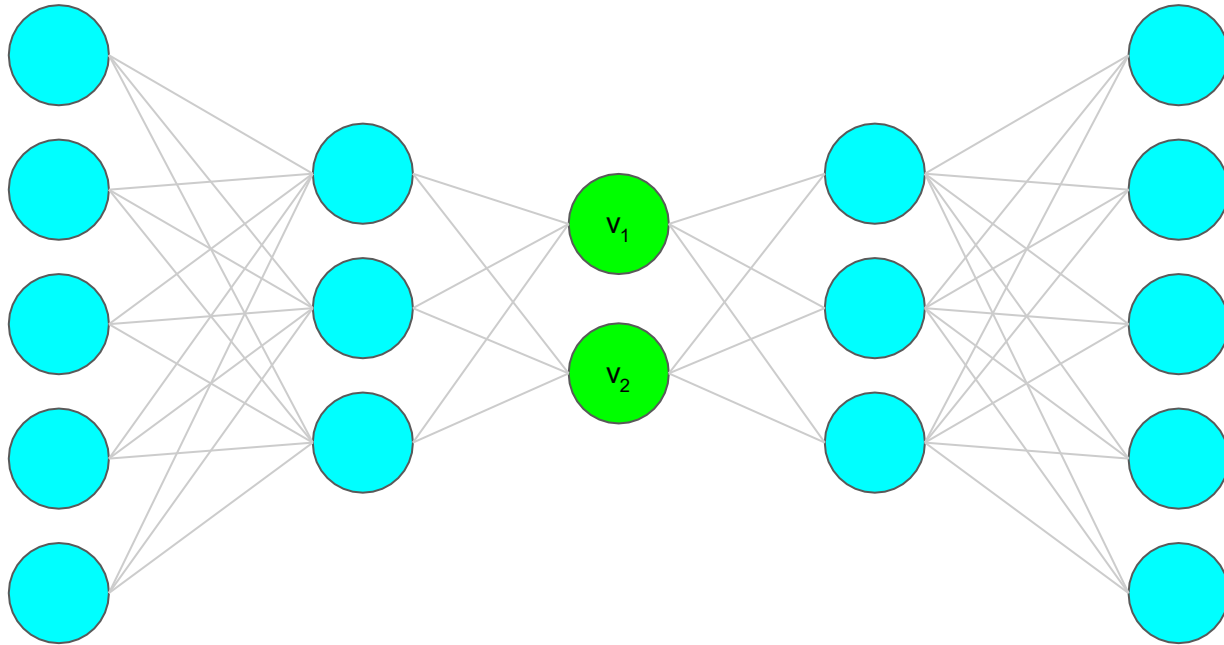
The addition of the KL divergence term ensures that the learned distribution is close to a prior distribution, which is essential for generating diverse and meaningful samples.



Generative AI models, **autoencoders (AE)** and **variational autoencoders (VAEs)** have emerged as powerful unsupervised learning techniques for data representation, compression, and generation



6-Deep Autoencoder



6-Deep Convolutional Autoencoder (CAE)

- Similar architecture to AE
- Convolutional layers
- **Encoder:** Convolution + Leaky Relu +Batch normalization
- **Decoder:** Convolution transpose + Leaky Relu + Batch normalization

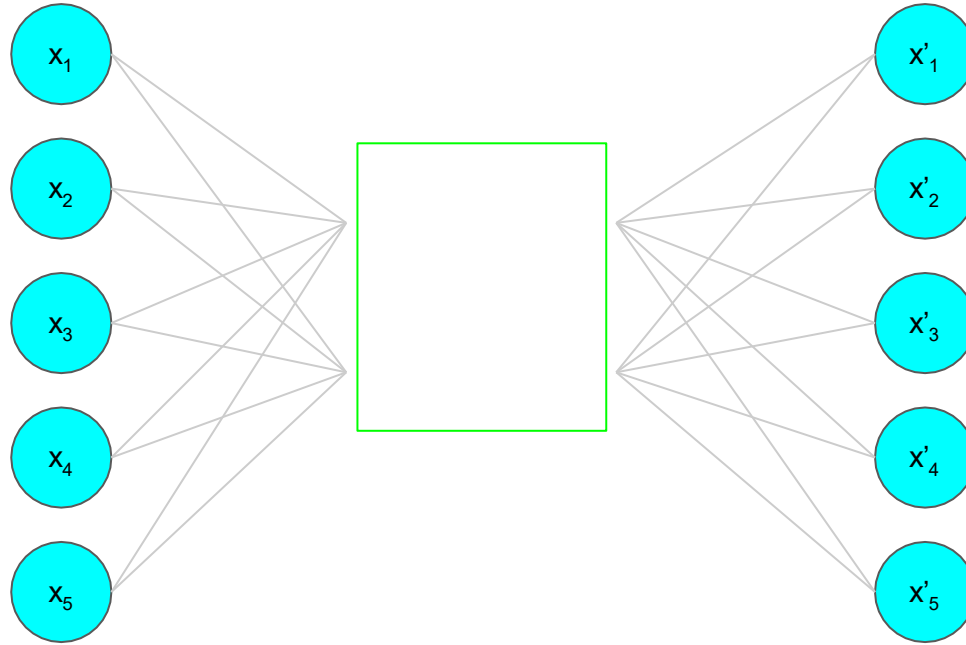
Deep Autoencoder Advantages



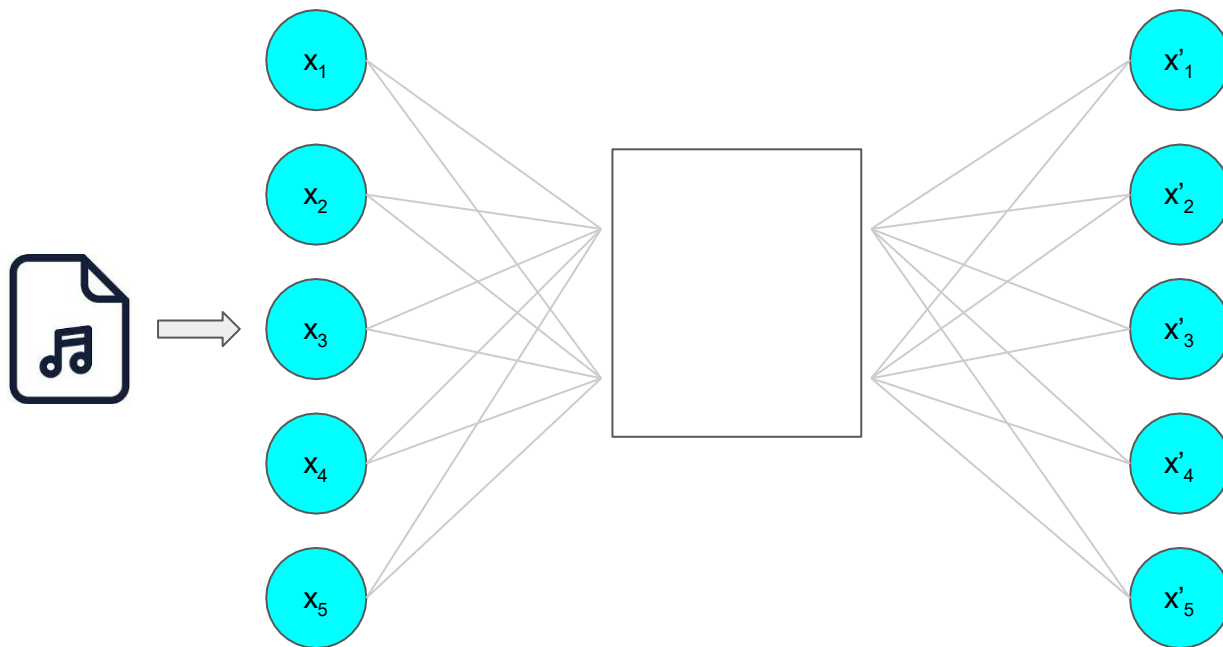
- **Image denoising:** CAEs can filter out noise and redundant information from images.
- **Dimensionality reduction:** CAEs can compress high-dimensional images into a lower-dimensional representation.
- **Image generation:** CAEs can reconstruct images from a lower-dimensional representation.
- **Defect detection:** CAEs can be used to detect defects in artworks by enhancing thermal images and extracting features.
- **Subsurface flow property distribution:** CAEs can be used to parameterize complex subsurface flow property distributions.
- **ECG denoising and classification:** CAEs can be used to denoise and classify electrocardiogram (ECG) signals.

CAEs are powerful tools that can be applied in many industries, including healthcare, anomaly detection, information retrieval, and recommendation systems

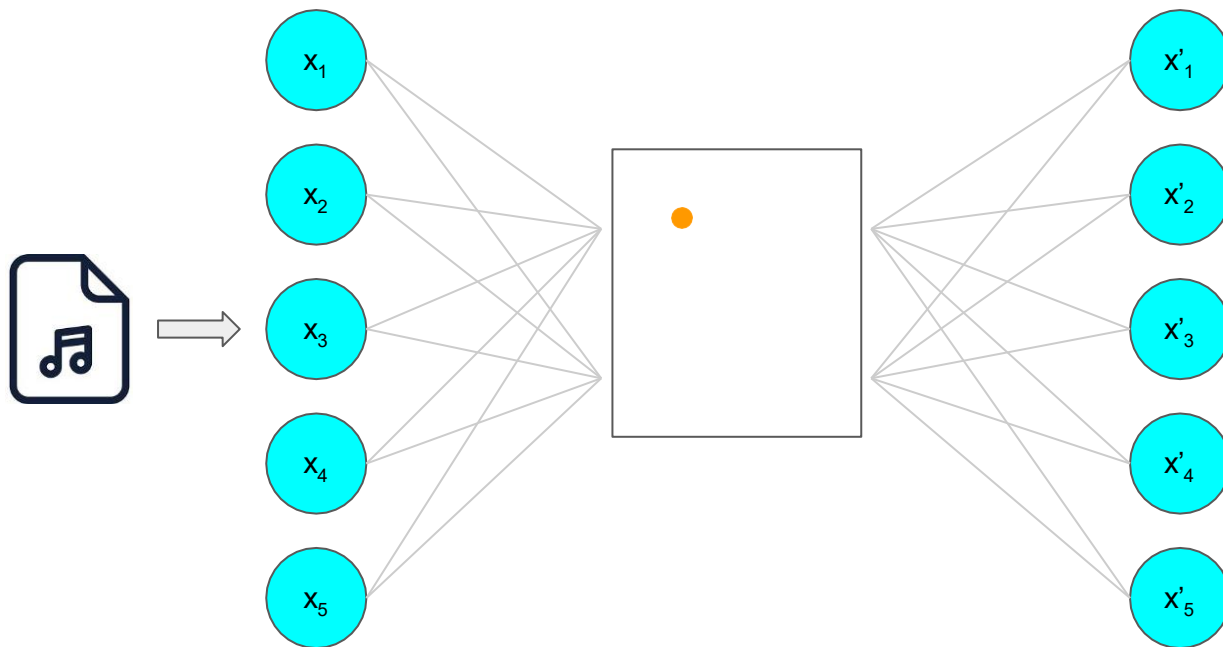
Generation with AEs



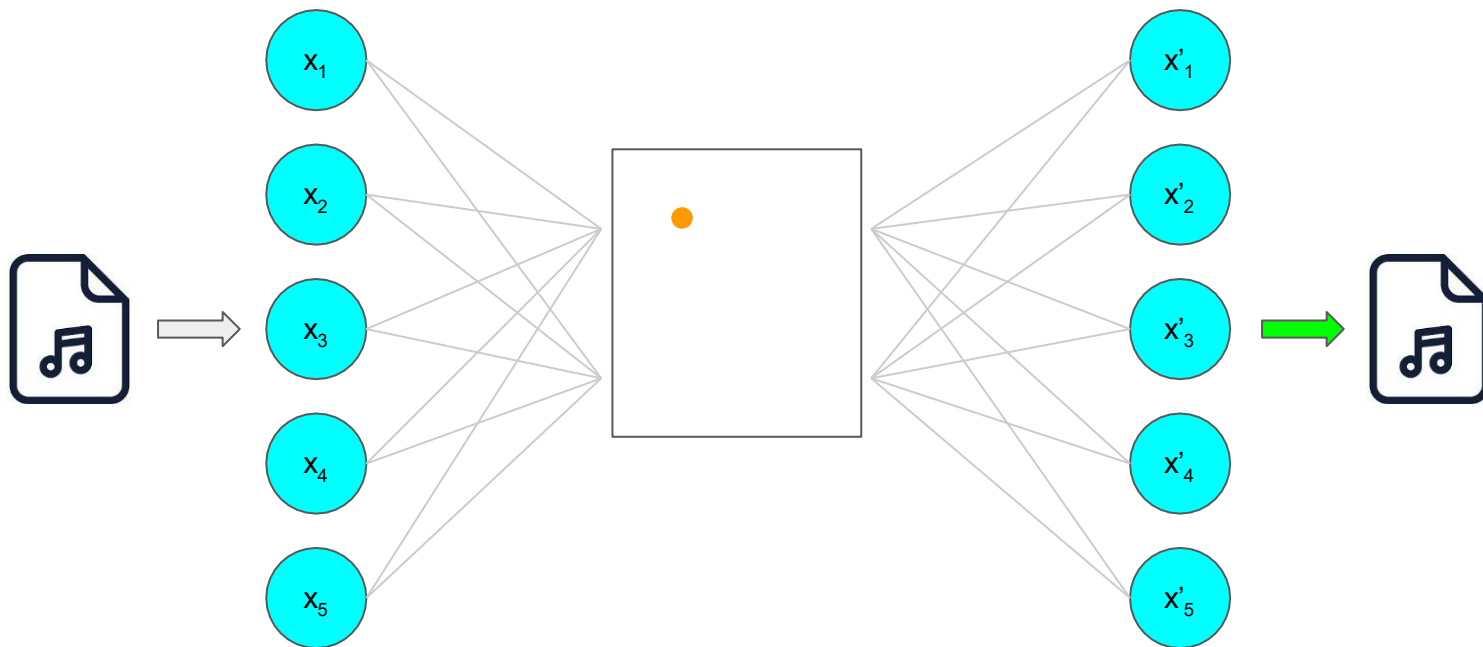
Generation with AEs



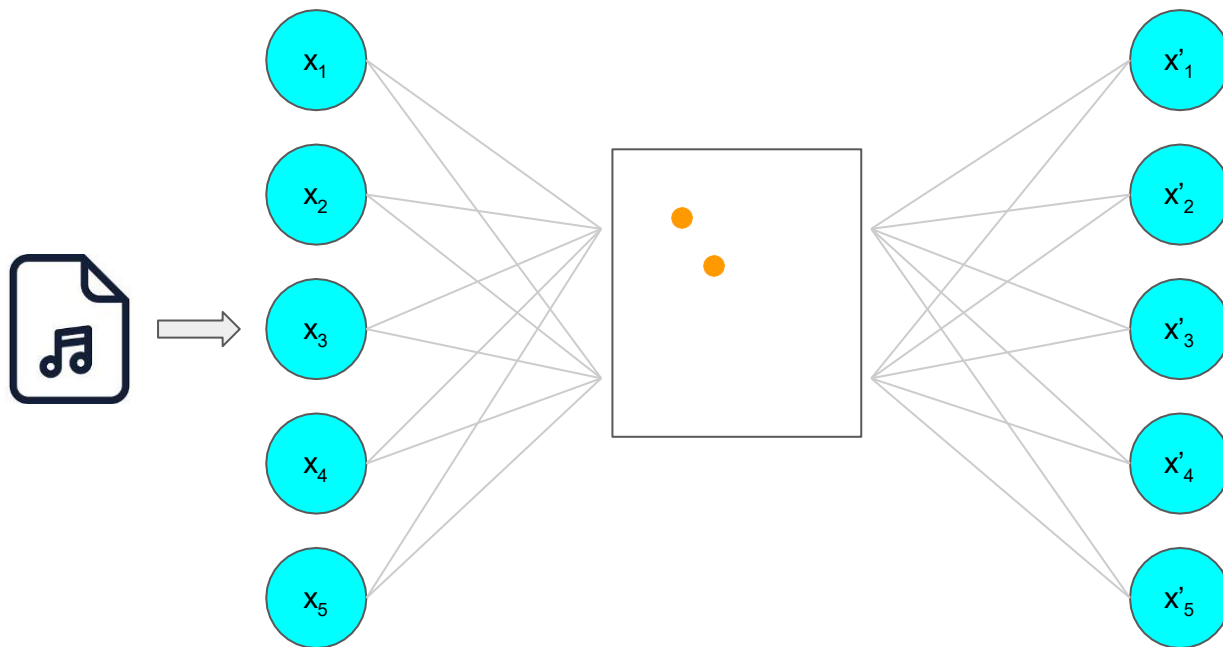
Generation with AEs



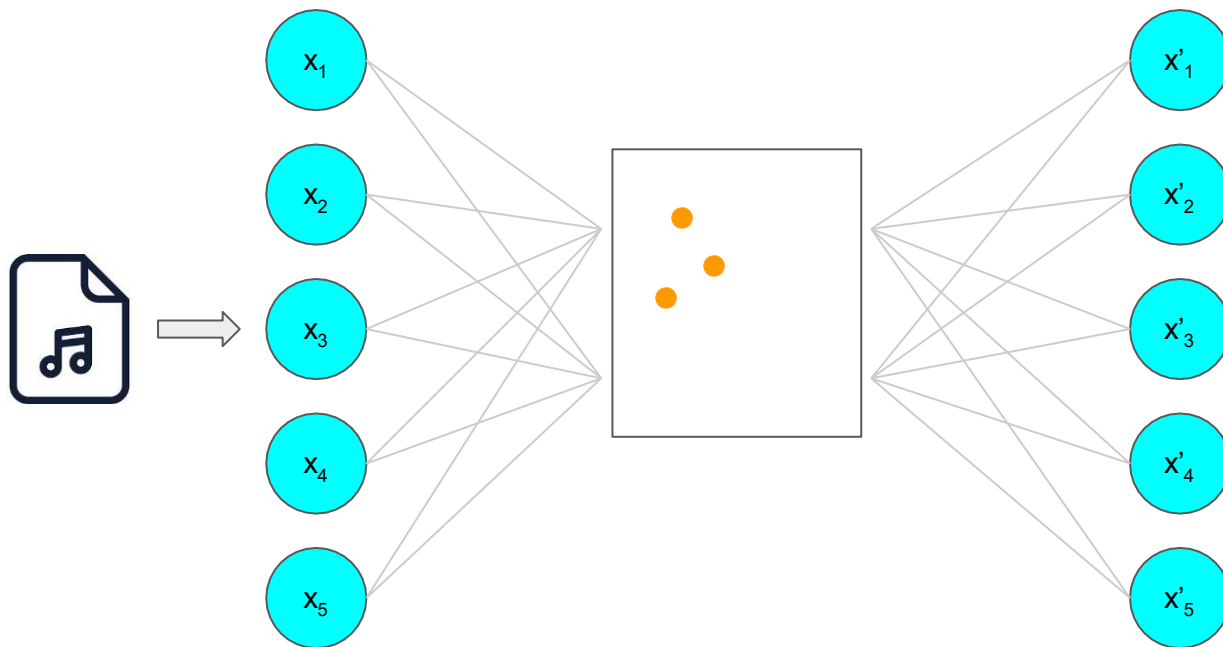
Generation with AEs



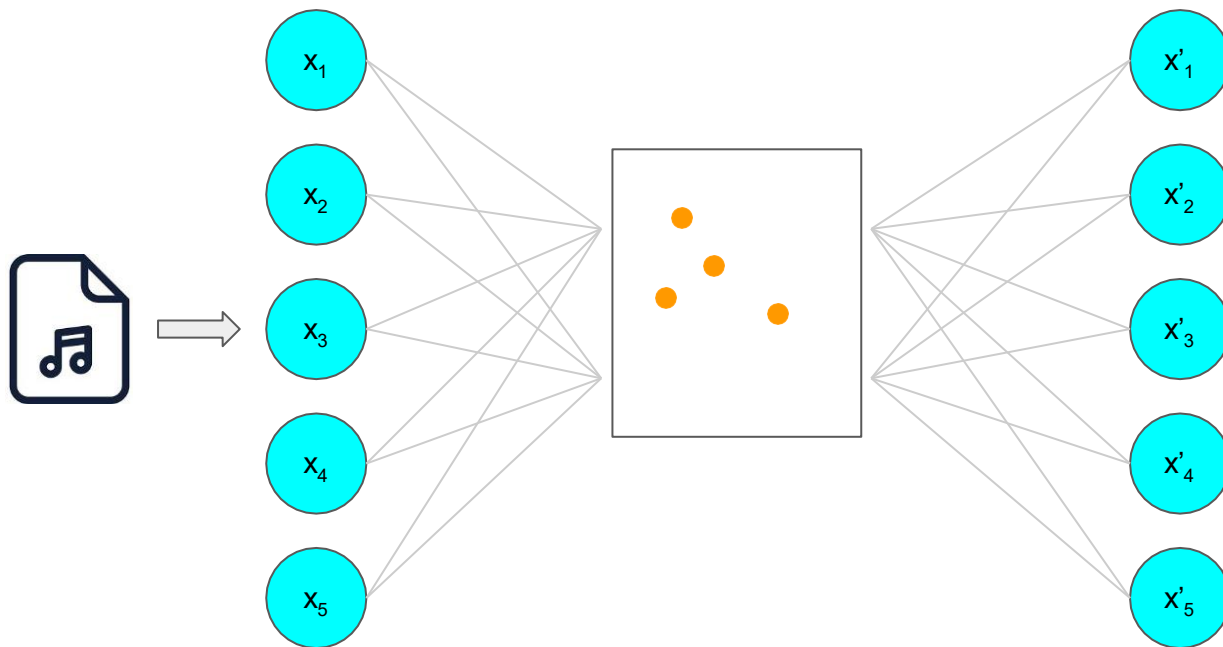
Generation with AEs



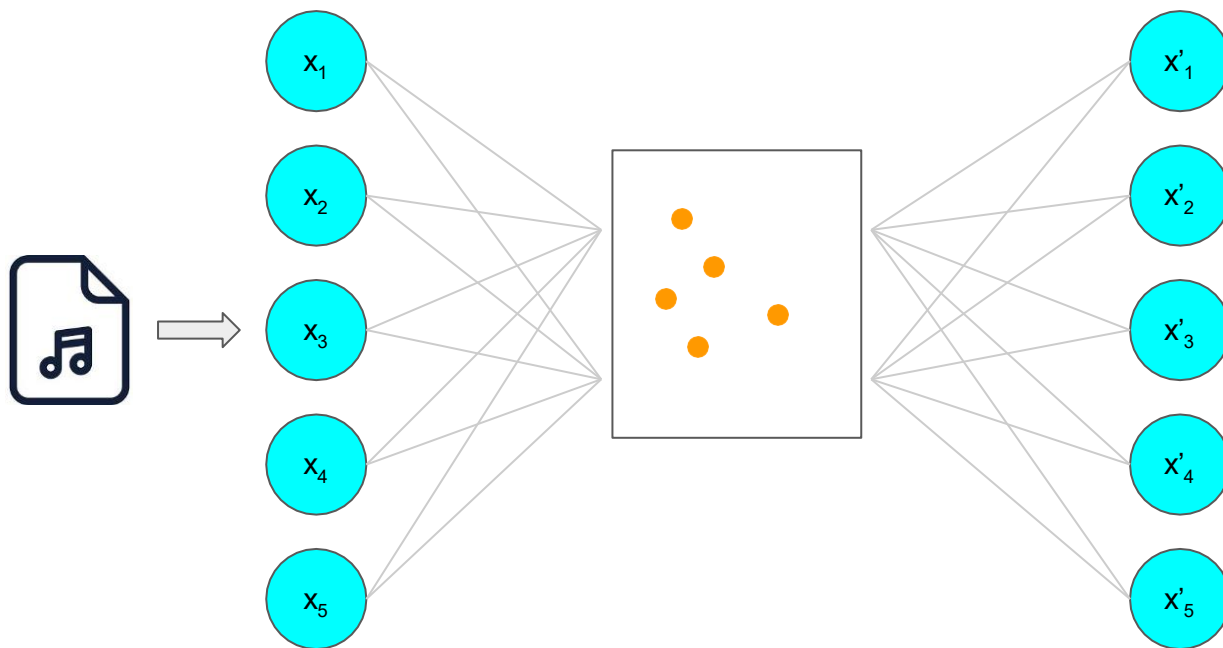
Generation with AEs



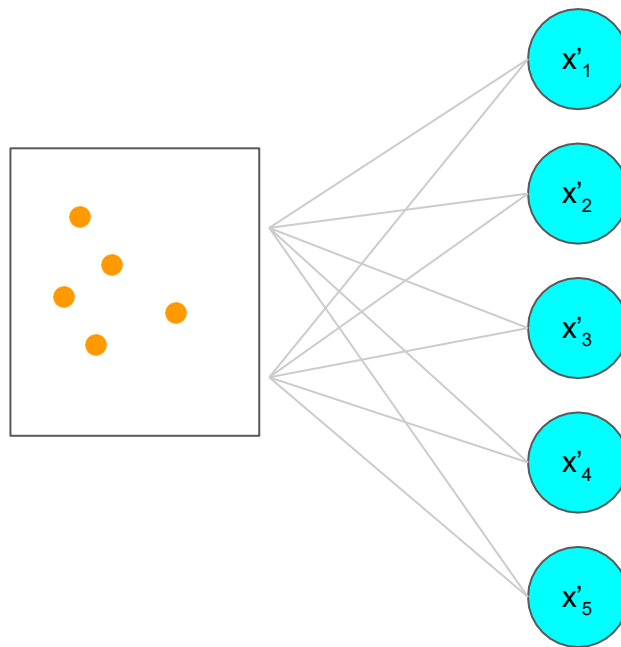
Generation with AEs



Generation with AEs

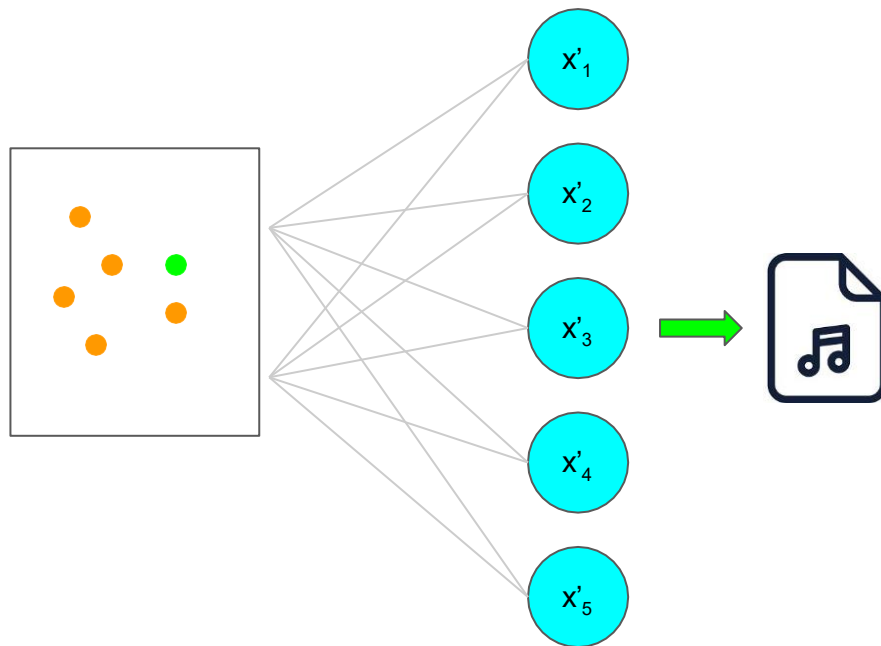


Generation with AEs

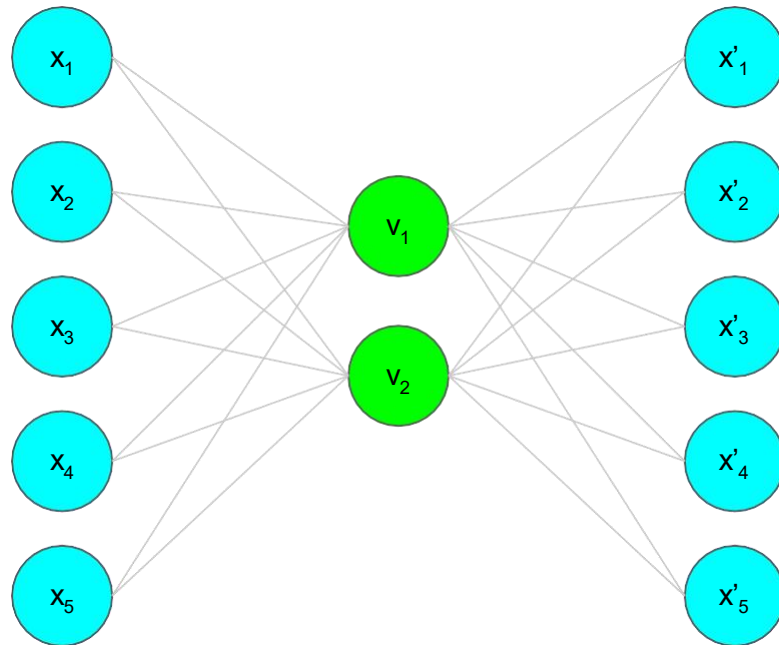


Generation with AEs

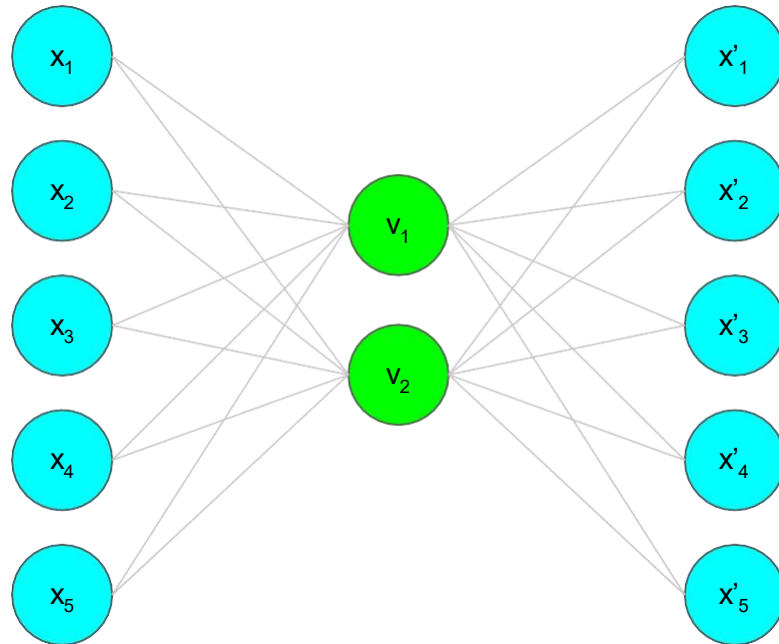
Sample a point in the latent space and pass it through the decoder



Denoising with AEs



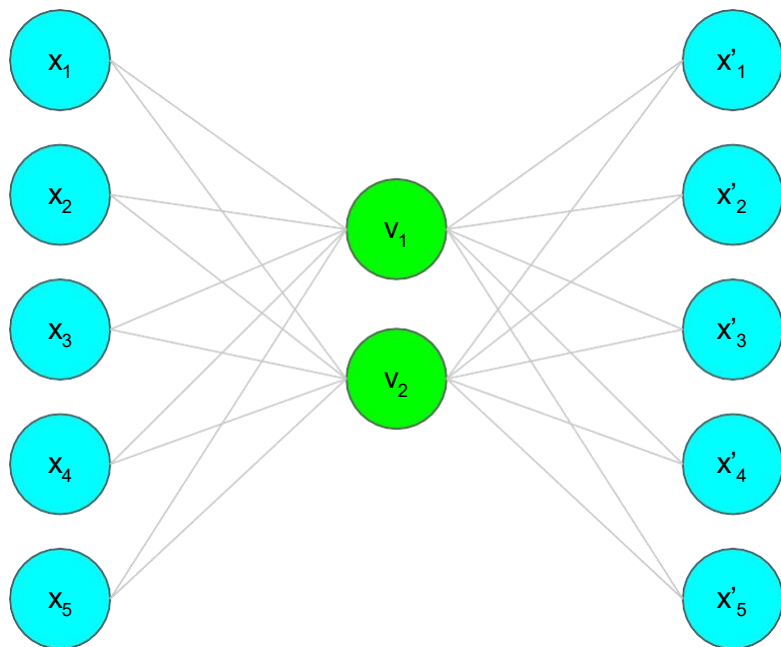
Denoising with AEs



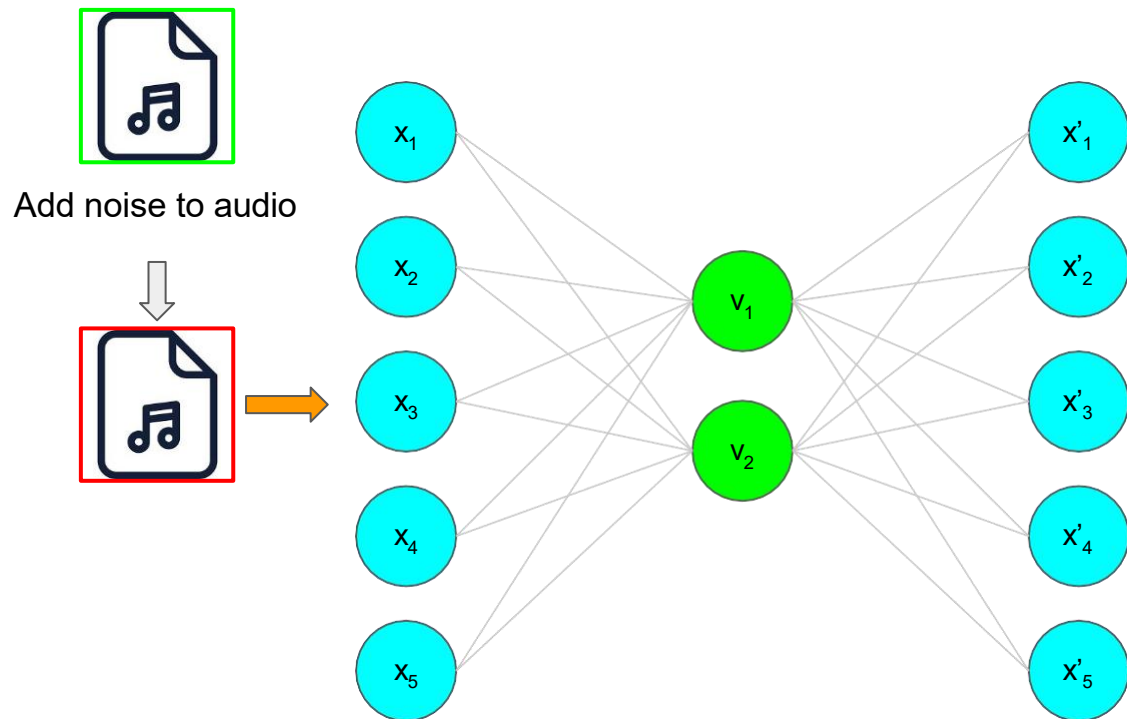
Denoising with AEs



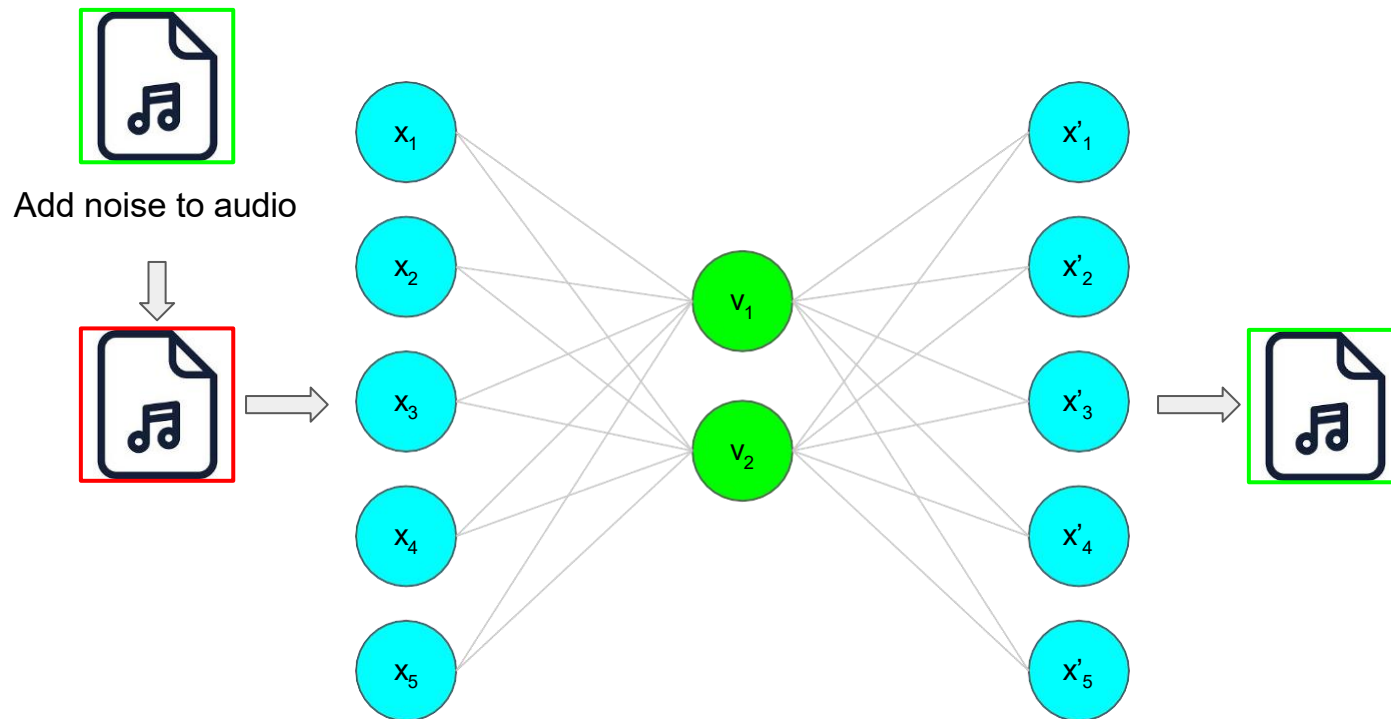
Add noise to audio



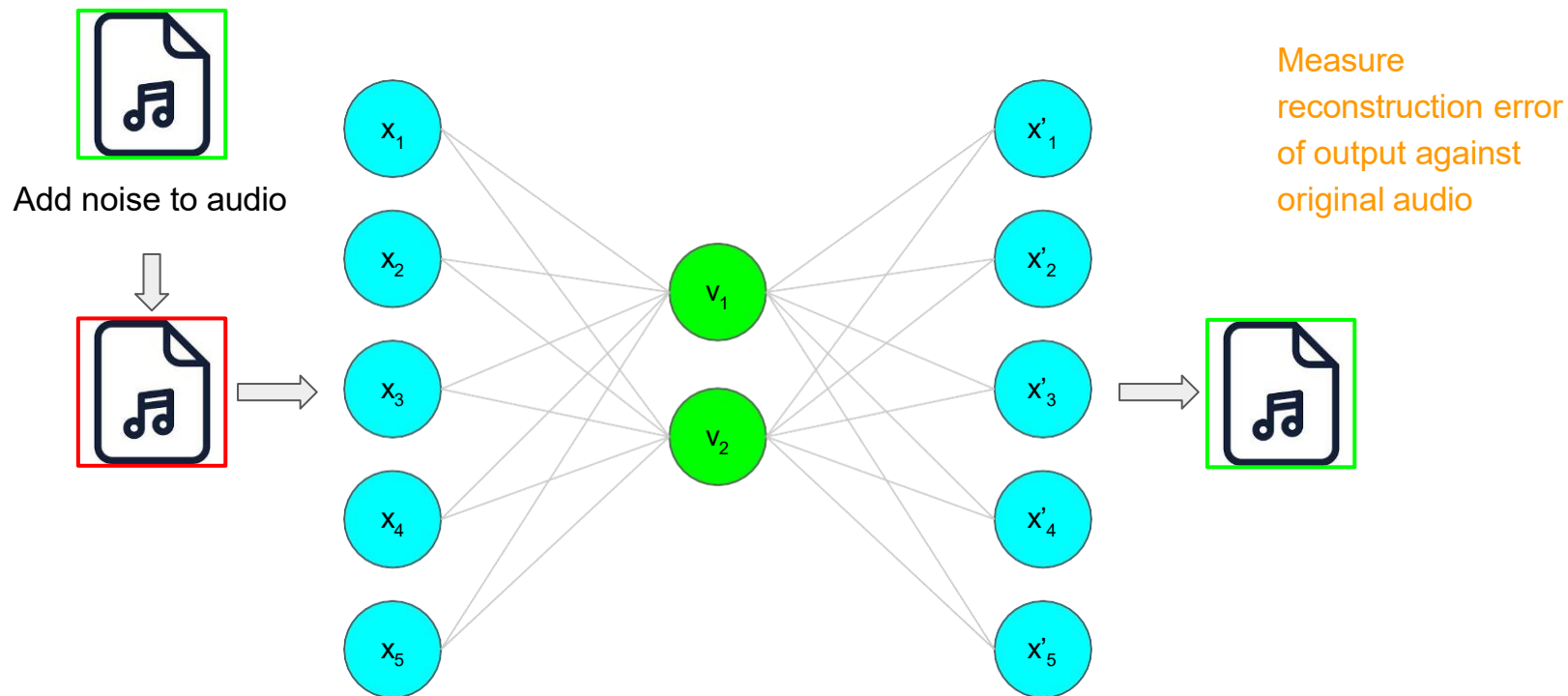
Denoising with AEs



Denoising with AEs



Denoising with AEs

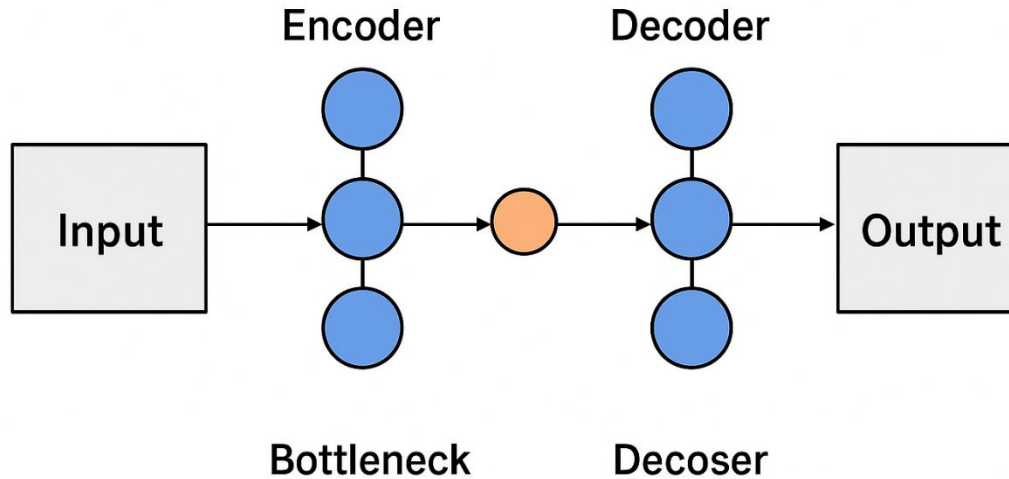


Autoencoder doesn't see the clean image as input — it *learns* to restore it.



Stage	Input	Output	Description
Encoder	Noisy image (784 features)	Compressed features (32)	Learns <i>noise-robust representations</i>
Decoder	Encoded representation	Denoised image	Learns to reconstruct clean image
Loss Function	Binary Crossentropy	Measures how close reconstruction is to clean input	

DENOISING AUTOENCODER





```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()

# Normalize to [0,1]
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.

# Flatten images (28x28 → 784)
x_train = x_train.reshape((len(x_train), 28 * 28))
x_test = x_test.reshape((len(x_test), 28 * 28))
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 ————— 1s 0us/step



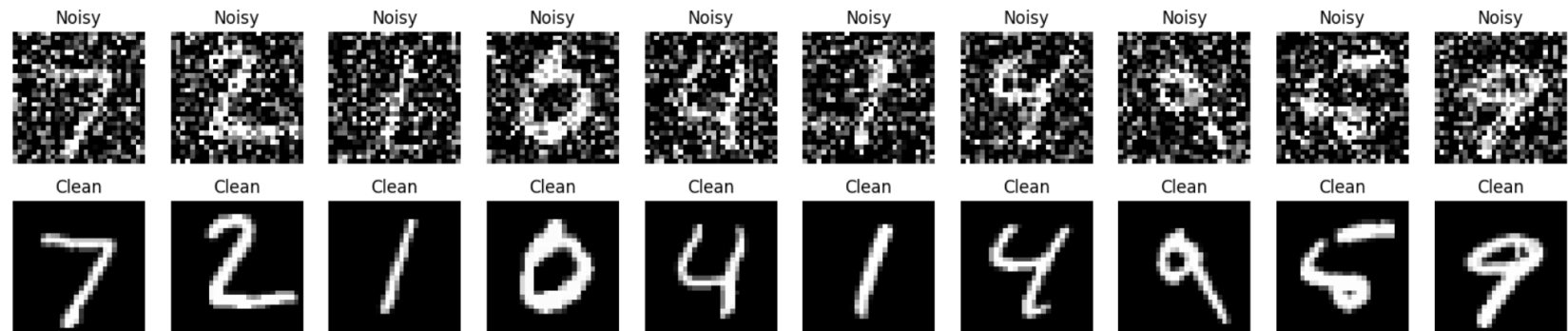
```
# Add Gaussian noise
noise_factor = 0.5
x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)

# Keep pixel values within [0, 1]
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)
```

```
n = 10
plt.figure(figsize=(20, 4))
for i in range(n):
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap='gray')
    plt.title("Noisy")
    plt.axis("off")

    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title("Clean")
    plt.axis("off")
plt.show()
```

```
plt.show()
```





[5]
✓ 0s

```
input_img = layers.Input(shape=(784,))

# Encoder
encoded = layers.Dense(128, activation='relu')(input_img)
encoded = layers.Dense(64, activation='relu')(encoded)
encoded = layers.Dense(32, activation='relu')(encoded)

# Decoder
decoded = layers.Dense(64, activation='relu')(encoded)
decoded = layers.Dense(128, activation='relu')(decoded)
decoded = layers.Dense(784, activation='sigmoid')(decoded)

# Model
autoencoder = models.Model(input_img, decoded)

autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
```

[6]
✓ 1m

```
autoencoder.fit(
    x_train_noisy, x_train,
    epochs=20,
    batch_size=256,
    shuffle=True,
    validation_data=(x_test_noisy, x_test)
)
```

Epoch 1/20

```

235/235 _____ 4s 16ms/step - loss: 0.1502 - val_loss: 0.1457
Epoch 6/20
235/235 _____ 5s 20ms/step - loss: 0.1454 - val_loss: 0.1418
Epoch 7/20
235/235 _____ 4s 16ms/step - loss: 0.1413 - val_loss: 0.1393
Epoch 8/20
235/235 _____ 4s 17ms/step - loss: 0.1382 - val_loss: 0.1360
Epoch 9/20
235/235 _____ 5s 20ms/step - loss: 0.1356 - val_loss: 0.1344
Epoch 10/20
235/235 _____ 4s 16ms/step - loss: 0.1338 - val_loss: 0.1324
Epoch 11/20
235/235 _____ 5s 16ms/step - loss: 0.1322 - val_loss: 0.1309
Epoch 12/20
235/235 _____ 5s 22ms/step - loss: 0.1307 - val_loss: 0.1299
Epoch 13/20
235/235 _____ 4s 16ms/step - loss: 0.1294 - val_loss: 0.1287
Epoch 14/20

```



```
▶ decoded_imgs = autoencoder.predict(x_test_noisy)

# Display noisy vs denoised vs original
n = 10
plt.figure(figsize=(20, 6))
for i in range(n):
    # Noisy input
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap='gray')
    plt.title("Noisy")
    plt.axis("off")

    # Denoised output
    ax = plt.subplot(3, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.title("Denoised")
    plt.axis("off")

    # Original clean image
    ax = plt.subplot(3, n, i + 1 + 2*n)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title("Original")
    plt.axis("off")
plt.show()
```

313/313 — 1s 2ms/step

